



GitHub Copilot

Capítulo 2 – Uso práctico por entorno

Marzo 2026

Repaso – Capítulo 1

5 ideas clave de la sesión anterior



El LLM predice, no piensa

Tu contexto es su combustible. Más contexto = mejores respuestas.



Prompting estructurado

ROL + CONTEXTO + TAREA + RESTRICCIONES + FORMATO. Nada de "arréglame esto".



Agent Loop

Pensar → Actuar → Observar → Repetir. El agente itera hasta completar.



Instructions del repo

`.github/copilot-instructions.md` define las reglas del proyecto.



SIEMPRE revisar

Copilot amplifica tu criterio. La responsabilidad final es tuya.



Para los escépticos...



"La IA genera basura"



Genera lo que le pides. Mal prompt = mal resultado.



"Tardo más corrigiendo que haciéndolo yo"



Normal al principio. Hoy vamos a reducir esa curva.



"No me fío de código que no entiendo"



Perfecto. Revisarlo es parte del flujo, no un fallo.

Hoy: menos teoría, más teclado. Vamos a trabajar con vuestras tareas de Jira reales.

01



Copilot en tu entorno

Visual Studio · VS Code · CLI — Mismo cerebro, distintas interfaces

Tres entornos, un ecosistema

Elige el que encaje con tu flujo de trabajo — todos comparten instrucciones y modelos

Visual Studio

COBOL (extensión) · .NET nativo

- Inline completions (Tab para aceptar)
- Chat panel integrado (Ctrl+/, Ctrl+Q)
- Copilot Edits (multi-archivo)
- Agent mode (preview)
- Ideal para proyectos .NET grandes

VS Code

Universal · Ligero · Extensible

- Inline completions + Chat (Ctrl+I)
- Modos: Ask / Edit / Agent (Ctrl+Shift+I)
- @workspace, @terminal, @vscode, @github
- Prompt files + Custom Agents
- El más completo en features Copilot

Copilot CLI

Terminal · Agnóstico de IDE

- Agente completo en tu terminal
- Plan mode (Shift+Tab) antes de actuar
- Sub-agentes con /task
- MCP servers integrados
- Funciona en cualquier entorno

Los tres modos del Chat

Cada modo tiene un propósito distinto — elegir bien ahorra tiempo



ASK

Pregunta sin tocar código

Cuándo: Explorar, entender, planificar

```
"¿Cómo funciona este PERFORM  
VARYING?"
```

VS: Chat panel

VSC: Ctrl+Shift+I → Ask

CLI: Shift+Tab → Ask



EDIT

Modifica archivos seleccionados

Cuándo: Cambios puntuales,
refactoring

```
"Añade validación de campos  
nulos"
```

VS: Copilot Edits

VSC: Ctrl+Shift+I → Edit

CLI: —



AGENT

Planifica + ejecuta + valida

Cuándo: Features completas, bugs
complejos

```
"Implementa el endpoint de  
facturas"
```

VS: Agent (preview)

VSC: Ctrl+Shift+I → Agent

CLI: Shift+Tab → Agent

Regla: Empieza siempre en Ask para entender. Pasa a Agent solo cuando tengas claro qué pedir.

Atajos y referencias esenciales

Lo que necesitas memorizar para ser productivo desde hoy

Inline Completions (todos los IDEs)

Tab

Aceptar sugerencia completa

Esc

Rechazar sugerencia

Alt +] / [

Siguiente / anterior alternativa

Ctrl + →

Aceptar palabra a palabra

Ctrl + Enter

Ver panel de alternativas

Referencias de contexto (Chat)

#file

Referencia un archivo específico

#selection

Código seleccionado actualmente

@workspace

Busca en todo el proyecto

@terminal

Contexto del terminal / errores

@github

Issues, PRs, repositorios

Copilot CLI – Comandos clave

Shift+Tab

Cambiar Ask/Plan/Agent

/context

Ver qué archivos ve el LLM

ESC ESC

Deshacer última acción

/model

Cambiar modelo

Ctrl+T

Toggle razonamiento

@archivo

Referenciar archivo

02



Contexto y Modelos

Lo que el modelo ve, y cuál elegir para cada tarea

Gestionando la ventana de contexto

El modelo solo 've' lo que cabe en su ventana. Tú decides qué incluir.

Alimenta bien el contexto

- ✓ Abre los archivos relacionados antes de preguntar
- ✓ Usa #file para referenciar archivos específicos
- ✓ Selecciona código relevante antes de usar Chat
- ✓ Usa @workspace para búsquedas en el proyecto
- ✓ Incluye copybooks/interfaces/DTOs en el contexto
- ✓ Pon un comentario descriptivo antes de la función

Errores que ensucian el contexto

- ✗ Conversaciones interminables (>20 turnos) sin /new
- ✗ Pedir sin archivos abiertos = respuesta genérica
- ✗ Mezclar temas en una misma sesión de chat
- ✗ Copiar código gigante en el prompt (usa #file)
- ✗ Ignorar el modelo: no todos sirven para todo
- ✗ Olvidar /context en CLI para ver qué ve el LLM



Truco CLI: usa /context para ver exactamente qué archivos está enviando al modelo. En VS Code: mira la sección "Used" del chat.

¿Qué modelo uso?

No todos los modelos sirven para todo — y los premium cuestan requests

Modelo	Premium	Úsalo para...	Evítalo cuando...
GPT-4.1 / GPT-5 mini	Gratis ✓	Tareas rutinarias, completions, preguntas rápidas, boilerplate	Lógica de negocio compleja, refactoring grande
Claude Sonnet 4 / GPT-5.1	1x req	Código preciso, refactoring, COBOL, review, debug	Cosas simples (usa el gratuito)
Claude Opus 4.6	10x req ⚠	Proyectos enormes, análisis cross-file, 1M tokens	Cualquier tarea normal (carísimo en requests)
Gemini 3 Pro	1x req	Multi-modal, documentos extensos, imágenes	Código .NET/COBOL puro (mejor Claude/GPT)



Recomendación para GCO

Día a día: GPT-5 mini (gratis). Cuando no te convenza la respuesta: cambia a Claude Sonnet 4 o GPT-5.1. Para análisis de proyecto completo o COBOL legacy muy grande: Claude Opus 4.6 (con cabeza, que cuesta 10x).

03



Modo Agéntico

De 'sugerirme código' a 'implementa esta tarea completa'

El Agent Loop en la práctica

El agente no ejecuta un script fijo — decide su próximo paso según cada resultado



Lo que el agente PUEDE hacer

- ✓ Leer y escribir archivos de tu proyecto
- ✓ Ejecutar comandos (build, test, lint)
- ✓ Buscar en el codebase (grep, AST)
- ✓ Crear branches, commits, PRs

Sistema de aprobación

Por operación

(default) Apruebas cada acción

/allow-all

Auto-aprueba en la sesión

/yolo

Sin confirmación (¡con precaución!)

El flujo correcto: Ask → Plan → Agent

No saltes directamente a Agent. El orden importa para obtener buenos resultados.

1 ASK

Entiende primero

Abre los archivos relevantes

Pregunta: "¿cómo funciona este módulo?"

Pide que te explique el código existente

Identifica dependencias y reglas

2 PLAN

Diseña el cambio

Shift+Tab → Plan mode (CLI)

Describe la tarea con contexto completo

El agente analiza y propone pasos

TÚ revisas el plan antes de ejecutar

3 AGENT

Ejecuta con supervisión

Aprueba el plan → el agente implementa

Revisa cada cambio propuesto

Pide correcciones en la misma sesión

Corre tests antes de dar por bueno

Para escépticos: el modo Agent no es "dejar que la IA haga lo que quiera". Es un flujo supervisado con puntos de control.

04



Personalización

Instructions, Prompt Files y configuración del agente

El sistema de personalización

Tres niveles que se combinan para que Copilot trabaje según TUS reglas

Instructions

```
.github/copilot-instructions.md
```

Reglas permanentes del repositorio. Se aplican SIEMPRE a cada prompt, en cada entorno.

Ejemplo:

"Usa C# 12 con .NET 8. Naming: PascalCase. Tests con xUnit. Comenta en español."

Prompt Files

```
.github/prompts/*.prompt.md
```

Plantillas reutilizables que invocas con /nombre. Control de versiones incluido.

Ejemplo:

generate-tests.prompt.md → Invoca con /generate-tests en el chat

Personal (IDE)

```
Settings > Copilot > Instructions
```

Tus preferencias personales. Máxima prioridad. No se comparten con el equipo.

Ejemplo:

"Responde siempre en español. Prefiero código conciso sin comentarios obvios."

Prioridad: Personal > Repository > Organization. Los tres se combinan en cada petición.

Ejemplo real: copilot-instructions.md

Este archivo va en la raíz de .github/ y aplica a TODOS los miembros del equipo

Proyecto .NET

Instrucciones para Copilot

Stack

- C# 12, .NET 8, Entity Framework
- xUnit para tests, FluentAssertions

Convenciones

- PascalCase para métodos y clases
- Inyección de dependencias siempre
- XML docs en métodos públicos
- Responde en español

Seguridad

- Nunca hardcodear secrets
- Validar siempre inputs del usuario

Proyecto COBOL

Instrucciones para Copilot

Entorno

- COBOL Enterprise, mainframe z/OS
- Copybooks en carpeta /copybooks
- JCL para ejecución batch

Convenciones

- Columnas 7-72 estrictamente
- Comentarios con * en columna 7
- Variables: prefijo según sección
- Responde en español

Importante

- Incluir siempre copybooks con COPY

Lo configuráis una vez → todo el equipo recibe las mismas instrucciones → consistencia garantizada

Ejercicio 1 – COBOL: Tu tarea real de Jira

Preparación (2 min)

Abre tu tarea de Jira. Baja el repositorio. Abre los fuentes en tu IDE.

Paso 1 – ASK: Entiende el código (5 min)

Abre los archivos COBOL + copybooks. Pregunta a Copilot:

"Explícame qué hace este programa y qué flujo sigue"

Paso 2 – PLAN: Describe tu cambio (5 min)

Copia la descripción del Jira. Cambia a Plan/Ask. Pide un plan de cambios.

Paso 3 – AGENT: Implementa (5 min)

Aprueba el plan. Deja que el agente implemente. REVISA cada cambio.

Paso 4 – VALIDA: ¿El resultado es correcto? (3 min)

Revisa el código generado. ¿Cumple la tarea? Itera si es necesario.

Ejercicio 2 – .NET: Tu tarea real de Jira

Preparación (2 min)

Abre tu tarea de Jira. Abre el proyecto .NET. Ten abiertos los archivos clave.

Paso 1 – ASK: Explora el código existente (5 min)

Usa @workspace o #file para que Copilot entienda el proyecto:

```
"@workspace ¿dónde se gestiona [tema de tu Jira]?"
```

Paso 2 – PLAN: Diseña la solución (5 min)

Pega la descripción del Jira. Pide un plan detallado de implementación.

Paso 3 – AGENT: Ejecuta y valida (5 min)

Modo Agent. Implementa. Pide que genere tests unitarios. Revisa.

Paso 4 – ITERA: Mejora el resultado (3 min)

¿Falta algo? Pide correcciones en la misma sesión. Prueba otro modelo.

Ejercicio 3 – COBOL: Debug con logs de Elastic

Preparación (2 min)

Busca una incidencia de Jira con error. Localiza el log en Elastic.

Paso 1 – Copia el JSON del error de Elastic

Pégalo directamente en el chat de Copilot junto con el programa COBOL abierto.

Paso 2 – ASK: ¿Qué está fallando?

"Analiza este error de producción y dime dónde está el problema en el código"

Paso 3 – AGENT: Implementa el fix

Si el diagnóstico es correcto, pasa a Agent para que implemente la corrección.

Paso 4 – Revisa el fix generado

¿Resuelve el error? ¿Introduce regresiones? Compara con el original.

Ejercicio 4 – .NET: Debug + Tests automáticos

Preparación (2 min)

Busca un bug de Jira o una tarea de mejora .NET. Abre los archivos afectados.

Paso 1 – ASK: Diagnóstica el problema (3 min)

Usa @workspace + pega el error/log. "¿Por qué falla este endpoint?"

Paso 2 – AGENT: Implementa la corrección (5 min)

Modo Agent. Que implemente el fix y genere test unitario que cubra el caso.

Paso 3 – Pide que ejecute los tests (3 min)

"Ejecuta dotnet test y corrige cualquier error hasta que pasen todos"

Paso 4 – Revisa: código + test + resultado

¿El test es significativo? ¿El fix tiene sentido? ¿Rompió algo más?

Cuándo NO usar Copilot

Saber sus límites es lo que genera confianza real en la herramienta

No lo uses para...

- ✗ Decisiones de arquitectura críticas (tu criterio)
- ✗ Código con datos reales de producción (GDPR)
- ✗ Validar lógica de negocio (no conoce tu dominio)
- ✗ Reemplazar code review humano
- ✗ Secrets, passwords, connection strings en el chat

Donde realmente brilla

- ✓ Entender código legacy que no conoces
- ✓ Boilerplate, mappers, DTOs, CRUDs repetitivos
- ✓ Generar tests unitarios desde código existente
- ✓ Refactoring guiado (SOLID, clean code)
- ✓ Documentación y comentarios consistentes



La confianza se construye con uso honesto

No te vendemos que Copilot es perfecto. Te decimos que, bien usado, te ahorra horas de trabajo tedioso y te deja centrarte en lo que importa: pensar.

En resumen...



Elige tu entorno

VS Code para lo más completo, CLI para agilidad, Visual Studio para .NET nativo. Todos comparten instrucciones.



Ask → Plan → Agent

No saltes al agente. Entiende, planifica, ejecuta. El orden multiplica la calidad.



Alimenta el contexto

Abre archivos, usa #file y @workspace. El LLM no adivina lo que no le das.



Modelo según tarea

GPT-5 mini para lo diario, Claude Sonnet para precisión, Opus solo para lo épico.



Configura tu repo

copilot-instructions.md + prompt files
= consistencia para todo el equipo.



Revisa siempre

Copilot es tu copiloto. Tú conduces, tú decides, tú eres responsable.

" La mejor manera de confiar en una herramienta es entender cuándo funciona y cuándo no. "

¿Preguntas?

¡Gracias!